



DevHex

The background features a light blue gradient with abstract circuit-like patterns. Purple and orange lines, some straight and some curved, crisscross the frame. Small circles, some solid and some hollow, are placed at various points along these lines. In the bottom right corner, there is a cluster of blue dots and a series of overlapping blue squares.

Finsite Technical Progress

By **DevHex**



01

Business Adjustments

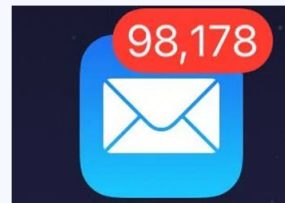


Our Competition

- Mainstream competitors are all cloud-based
- Most have pricey subscriptions

Many users are left looking for:

- Privacy-first options
- Offline usage
- Cheap alternative to a subscription

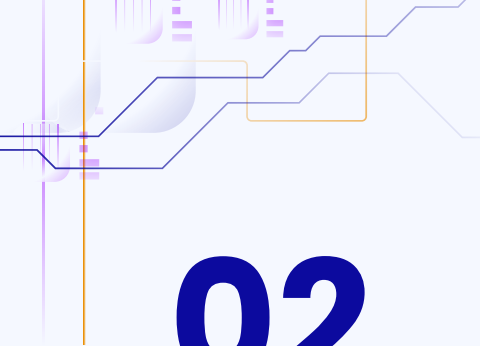


Updated Business Model

- Cloud-based → fully local using IndexedDB
- No account creation/login required
- Small one time upfront payment

Options to upload expenses:

- Manual
- CSV
- Email
- No bank linking
- No photo capture



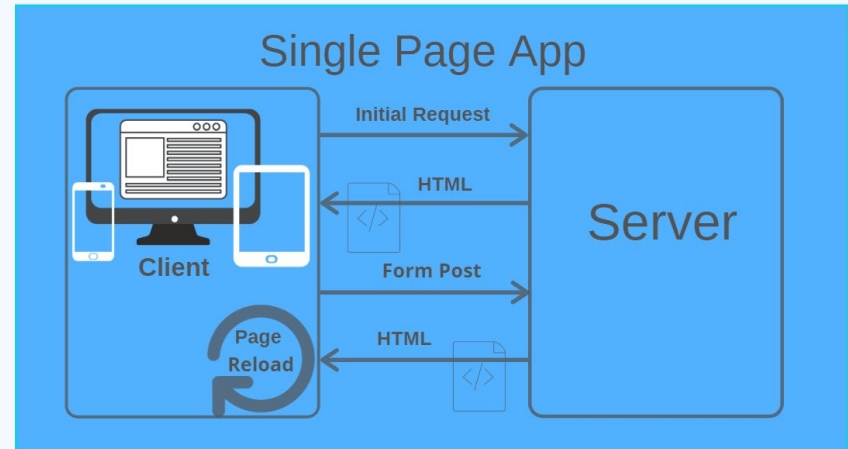
02

User Interface



User Interface Technology

- HTML/CSS/JavaScript
- Single Page Application with client-side routing
- Every “page” is a separate web component
- View in MVC is in charge of rendering the different components



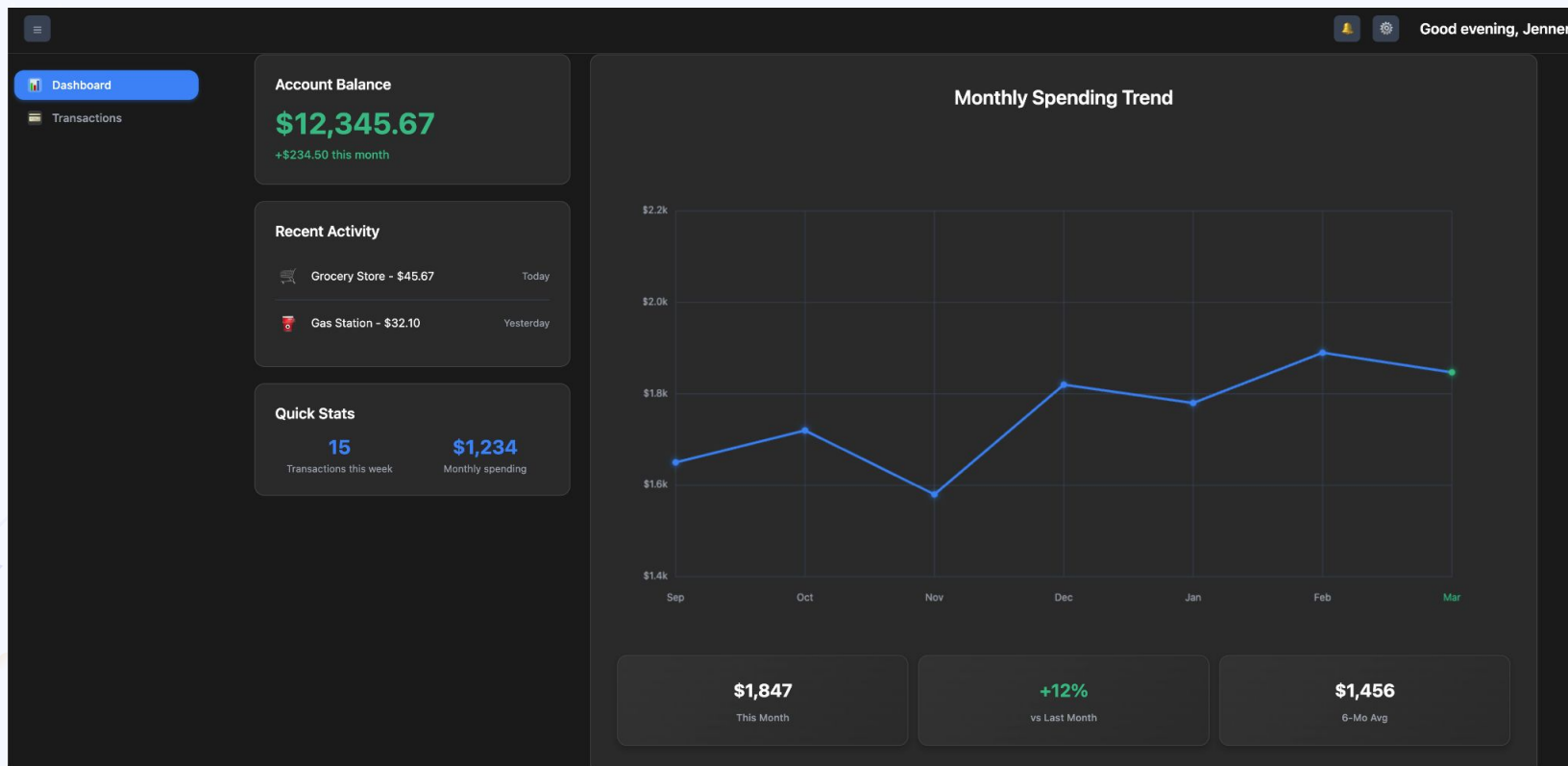
User Interface Summary

- Dashboard
 - Input Form: User enters category, amount, and date of expense
 - Graphs and tables summarizing spending
 - Link to categories page
- Category Selection Page
 - Links to a specific spending category page
- Specific Category Pages
 - Charts and tables summarizing spending in specific categories

UI Concept Preview

The background features a light blue gradient with various decorative elements. In the top-left corner, there are stylized circuit lines in purple, blue, and orange. The bottom-left corner contains a grid of small blue dots and a blue geometric shape. The right side of the image is decorated with a grid of blue dots and a complex network of circuit lines in purple, blue, and orange, including a wavy line and a circular node.

Dashboard





03

Storage



IndexedDB Overview

- Stores large amounts of structured data
- Works asynchronously (non-blocking)
- Supports objects, arrays, files, and complex records
- Data persists between sessions (saved on disk)
- Designed for data-heavy apps



IndexedDB vs LocalStorage

Feature	IndexedDB	LocalStorage
Data Size	~2GB	~5MB
Data Types	Objects, arrays, files etc	Strings only
Performance	Fast, async, non-blocking	slow , synchronous, blocks UI
Structure	Database,stores,records	Key-Value only
Memory Use	Loads only what we request	Loads entire values into RAM
Best for	Real apps with a lot of data	Small settings

Updates

- **Defined and documented a transaction object model**
 - Transaction objects had inconsistent structure
 - Now associated with ID, color / icon, appropriate relationships, and other attributes
- **Defined Groups and Category Entities**
 - In the process of replacing hard coded data with data retrieved from the database using distinct objects / payload to dynamically display information
 - Groups: household, investments, expenses
 - Currently have 7 categories across all groups in addition to custom categories
- **Model Layer**
 - Replacing hard coded data and adding structure to the model
 - Working on populating data and data attributes via `init()` after retrieving information from IndexedDB

Updates

- **IndexedDB Schema**

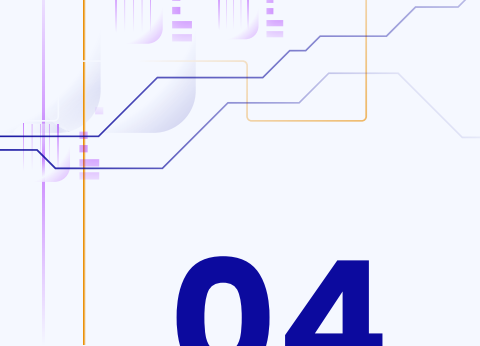
- Unique individual stores allow overall better performance in comparison to 1 store
- **Scalable**: Able to add more stores for other purposes (i.e. budgets or goals)
- **Faster queries**: Focused stores are faster to query than one giant store
- **Clear Domain Model**: Makes code easier to understand and maintain

- **Performance Optimization**

- $O(n) \rightarrow O(1)$
- Recounting every entry vs. Keeping a tab
- Live updates by keeping running totals and updating using individual entries instead of full recalculations everytime.
- Recalculation $O(n)$ happens on `init()`, Bulk import / CSV, Clearing ALL transactions
- Incremental Aggregation: Updating piece by piece instead of recalculating every time

Updates

- **Slow way (Before):**
 - You have a piggy bank with 100 coins
 - Every time someone asks "how much money do you have?", you dump out all 100 coins and count them one by one
 - When you add 1 more coin, you dump out all 101 coins and count again
- **Fast way (After):**
 - You count the 100 coins once at the start: \$50
 - Write "\$50" on a sticky note on the piggy bank
 - When you add a quarter, just update the sticky note: $\$50 + \$0.25 = \$50.25$
 - When someone asks "how much?", just read the sticky note



04

Manual Entry



Merchant Field Support

- We added a dedicated merchant field and kept notes for comments only to simplify parsing and imports. Separating merchant from notes lets future CSV, receipt, or email parsing map directly to a merchant without scraping free-text.
- Model + Storage
 - Added `merchant` string to transaction schema
 - CRUD persists/returns `merchant`
- UI
 - Manual-entry form: dedicated merchant input next to group/category
 - Notes remain comment-only
- Compatibility
 - Existing transactions remain valid; merchant defaults empty

Add Transaction Button

- Manual entry will act as the gateway for any transaction entering the site. Anything parsed or imported will flow into this form, so we added a clear trigger from the Transactions page.
- Component
 - Added primary “Add Transaction” button to <finsite-transactions> header (alongside filters/export)
- Behavior
 - Click emits bubbling custom event: add-transaction
 - No data handling or view changes inside the component
- Integration
 - Parent can listen and open manual-entry UI in a later refactor

Q&A



Thanks !